WIA1006/WID3006 MACHINE LEARNING

Semester 2, Session 2025/2026 | Universiti Malaya

DATA PRE-PROCESSING

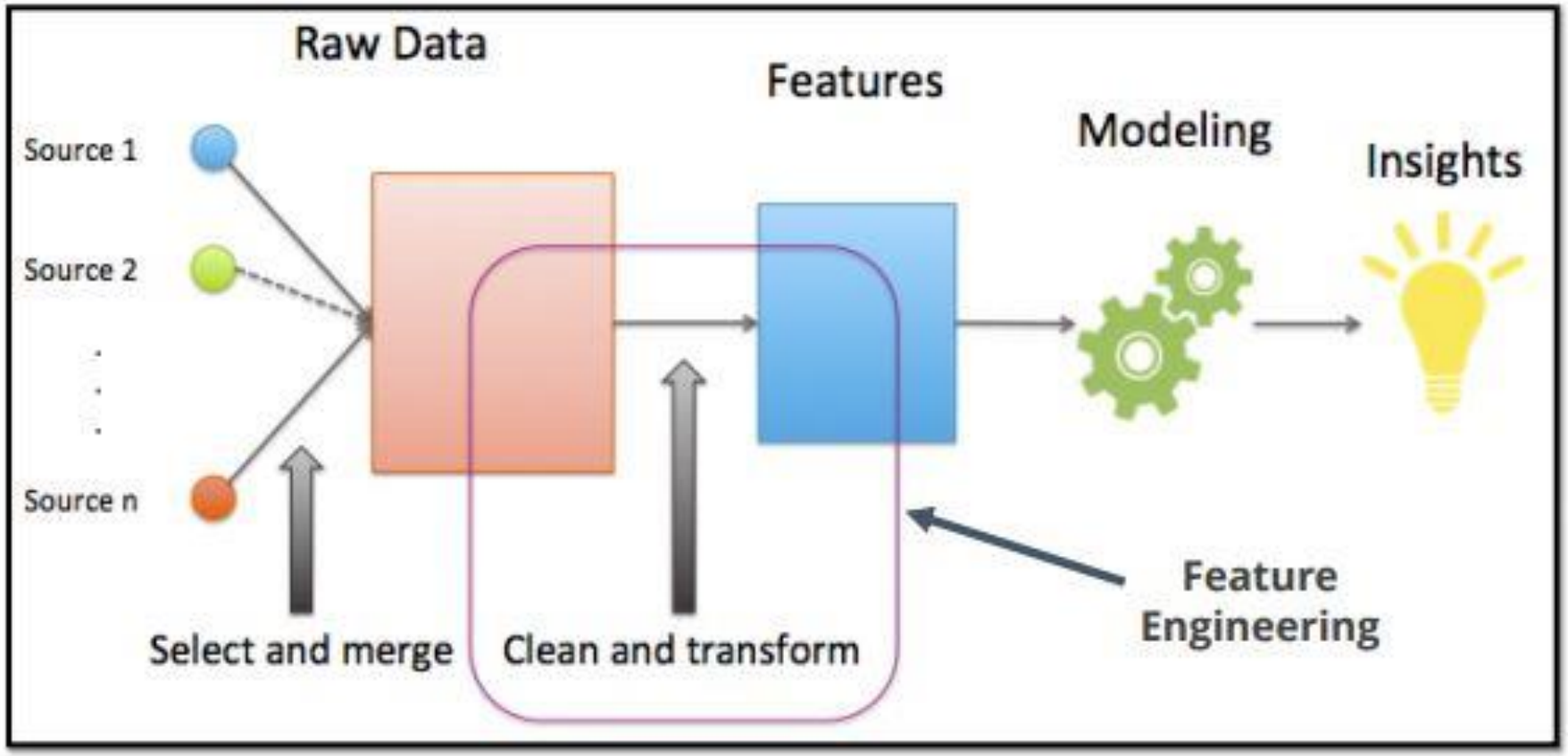
Data Pre-Processing & Cleansing

- Data pre-processing referring to transforming of data before feeding into the model
- **Converting unusable raw data into reliable and clean data**
- Raw data often contains:
  - Outliers
  - Meaningless data combination
  - Missing values
  - Scale problems

Data Pre-Processing & Cleansing

- Data preprocessing in Machine Learning is a crucial step that helps enhance the quality of data to promote the extraction of meaningful insights from the data.
- It refers to the technique of preparing (cleaning and organizing) the raw data to make it suitable for a building and training Machine Learning models.
- In simple words, data preprocessing in Machine Learning is a data mining technique that transforms raw data into an understandable and readable format.

Data Pre-Processing & Cleansing



Data Pre-Processing & Cleansing

7 Steps in data pre-processing:

1. Acquire the dataset
2. Import all the crucial libraries: matplotlib, numpy, pandas
3. Import the dataset
4. Identifying and handling missing values
5. Encoding the categorical data: dummy (), label encoder (), one hot encoder ()
6. Feature Scaling: minmax (), standardscaler()
7. Splitting the dataset: training (learning) , testing (validate)- Pareto Theory (80/20), (70/30), (60/40)

Data Pre-Processing & Cleansing

1. Acquire the dataset

Data gathering from multiple and disparate sources which are then combined in a proper format to form a dataset. Dataset formats differ according to use cases (for instance, a business dataset will be entirely different from a medical dataset. While a business dataset will contain relevant industry and business data, a medical dataset will include healthcare- related data).

2. Import crucial libraries

The predefined Python libraries can perform specific data preprocessing jobs. The three core Python libraries used for this data preprocessing are:

- NumPy - fundamental package for scientific calculation in Python
- Pandas - library for data manipulation and analysis.
- Matplotlib - 2D plotting library that is used to plot any type of charts in Python

Data Pre-Processing & Cleansing

3. Import the dataset

Before you can import the dataset/s, you must set the current directory as the working directory (see TutorialWeek02 and PracticalWeek02)

4. Identifying and handling missing values

In data preprocessing, it is pivotal to identify and correctly handle the missing values, failing to do this, you might draw inaccurate and faulty conclusions and inferences from the data.

Data Pre-Processing & Cleansing

5. Encoding categorical data

Machine Learning models are primarily based on mathematical equations. Thus, you can intuitively understand that keeping the categorical data in the equation will cause certain issues since you would only need numbers in the equations.

6. Feature scaling

- It is a method to standardize the independent variables of a dataset within a specific range.
- In other words, feature scaling limits the range of variables so that you can compare them on common grounds.

Data Pre-Processing & Cleansing

7. Splitting the dataset

Every dataset for Machine Learning model must be split into two separate sets:

1. training set
2. test set

Data Pre-Processing & Cleansing

Other common Pre-processing steps:

1. Data integration: process of combining data from multiple source systems to create unified sets of information for both operational and analytical uses.
2. Data Transformation: process of converting data from one format to another, typically from the format of a source system into the required format of a destination system.
3. Data Discretization: a process of converting continuous data attribute values into a finite set of intervals and associating with each interval some specific data value

Data Pre-Processing & Cleansing

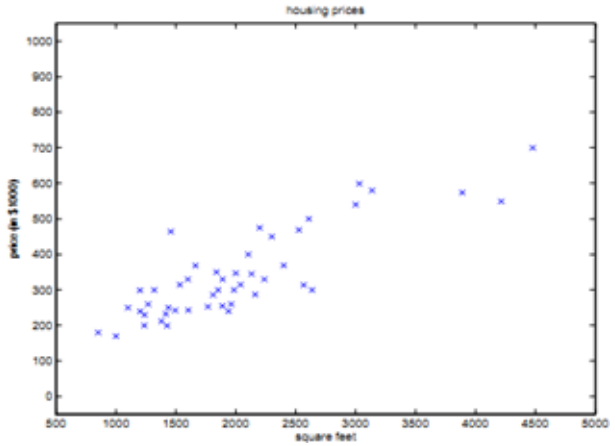
REGRESSION ANALYSIS

Supervised Learning

- Suppose we have a dataset giving the living areas and prices of 47 houses from SS17, Petaling Jaya
- Given data like this, how can we learn to predict the prices of other houses in SS17, as a function of the size of their living areas ?

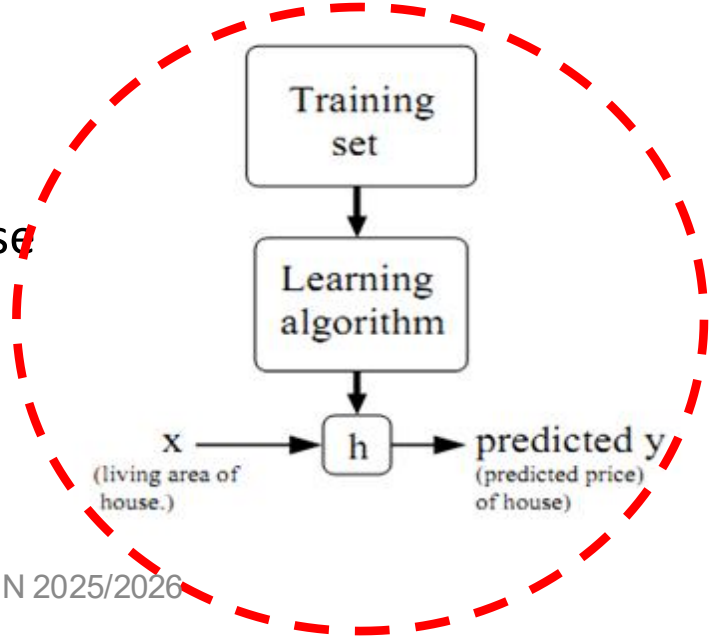
Living Area (feet^2)	price (RM 1K)
2104	400
1600	330
2400	369
1416	232
3000	540
⋮	⋮

We can plot this data:



Notation

- $x^{(i)}$ = input features (living area in this example)
- $y^{(i)}$ = output/target variable that we are trying to predict (in this example = price)
- A pair $(x^{(i)}, y^{(i)})$ = a list of *m* training example, where *i* = {1,…,….,m}
- To describe the supervised learning problem, our goal is given a training set, to learn a function, h: X➡ Y so that *h(x)* is a "good" predictor for the corresponding value of y.
- h = hypothesis
- When the target variable that we are trying to predict is continuous, such as in our house example, we call the learning problem a regression problem.



Linear Regression

- Let's consider a slightly richer dataset in which we also know the # of bedrooms in each house:

Living Area (feet²)	#bedrooms	Price (RM 1K)
2104	3	400
1600	3	330
2400	3	369
1416	2	232
3000	4	540
⋮	⋮	⋮

- Here, the x's are 2-dimensional vectors in $\mathbb{R}^2$. For instance, $x^{(i)}_1$ is the living area of *i*-th house in the training set, and $x^{(i)}_2$ is its number of bedrooms.
- In order to perform supervised learning, we must decide how we are going to represent functions/hypotheses, *h* in a computer. As an initial choice, let's say we decide to approximate *y* as a linear function of *x*:

$$h_\theta(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2$$

Linear Regression

$$h_\theta(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2$$

- Here, the θ_i are the parameters (also called weights) parameterizing the space of linear functions mapping for x → y.
- To simplify, we also introduce the convention of letting $x_0 = 1$, so that

$$h(x) = \sum_{i=0}^{n} \theta_i x_i = \theta^T x,$$

NOW, given a training set, how do you learn the parameter θ ??

Linear Regression

- SOLUTION:
  - **(English)** Make *h(x)* close to *y*, at least the training examples we have.
  - **(Mathematics)** Define a **function** that measures, for each value of the θs, how close the $h(x^{(i)})$'s are to the corresponding $y^{(i)}$'s. We define the **least-square cost function**:

$$J(\theta) = \frac{1}{2}\sum_{i=1}^{m}(h_\theta(x^{(i)}) - y^{(i)})^2.$$

Least Mean Square algorithm (LMS)

- We want to choose θ so as to minimise J(θ).
  - Initial Step

    Random "initial guess" for θ
  - 2nd Step

    Check the J(θ)
  - 3rd Step

    If J(θ) <<, change θ, else STOP
  - Repeat 3rd Step till converge to a value of θ that minimises J(θ).

Batch Gradient Descent Algorithm

$$\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta).$$

- α is called the **learning rate**.
- (English) This is a very natural algorithm that repeatly takes a step in direction of steepest decrease of *J*.
- (Mathematics) We have to work out what is the **Partial Derivative** term on the right hand side.

$$\begin{aligned}
\frac{\partial}{\partial \theta_j} J(\theta) &= \frac{\partial}{\partial \theta_j} \frac{1}{2} (h_\theta(x) - y)^2 \\
&= 2 \cdot \frac{1}{2} (h_\theta(x) - y) \cdot \frac{\partial}{\partial \theta_j} (h_\theta(x) - y) \\
&= (h_\theta(x) - y) \cdot \frac{\partial}{\partial \theta_j} \left(\sum_{i=0}^{n} \theta_i x_i - y \right) \\
&= (h_\theta(x) - y) \, x_j
\end{aligned}$$

Batch Gradient Descent Algorithm

- For a single training example, this gives the update rule

$$\boxed{\theta_j := \theta_j + \alpha \left(y^{(i)} - h_\theta(x^{(i)})\right) x_j^{(i)}}$$

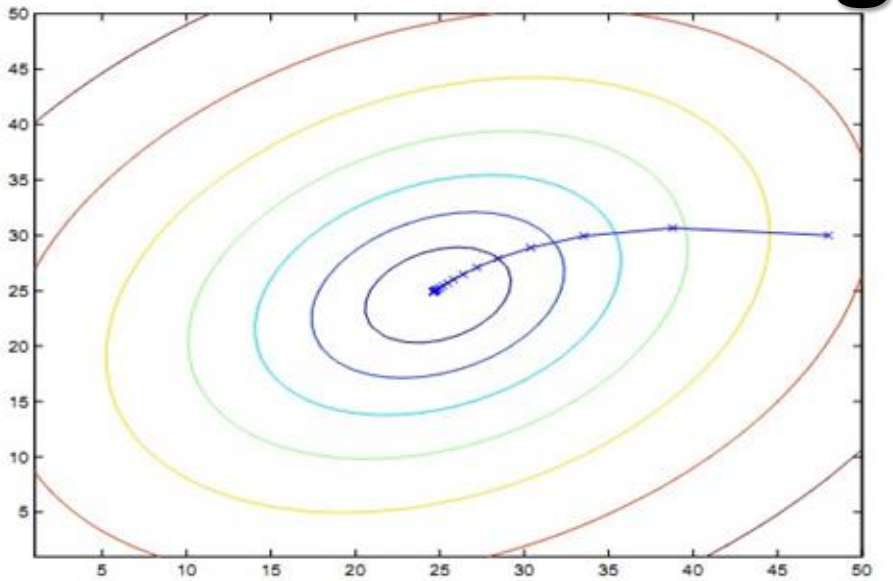
- The rule is called the **LMS** update rule, also known as **Widrow-Hoff** learning rule.
- The rule has several properties that seem natural and intuitive. For instance, the **magnitude of the update is proportional to the error term** $(y^{(i)} - h_\theta(x^{(i)}))$; thus when are encountering a training example on which our prediction ≈ actual value of y(i), then we find that there is little/no need to change the parameters; in contrast, a larger change to the parameters will be made if our prediction $h_\theta(x^{(i)})$ has a large error.

Repeat until convergence {

$$\theta_j := \theta_j + \alpha \sum_{i=1}^{m} \left(y^{(i)} - h_\theta(x^{(i)})\right) x_j^{(i)} \qquad \text{(for every } j\text{).}$$

}

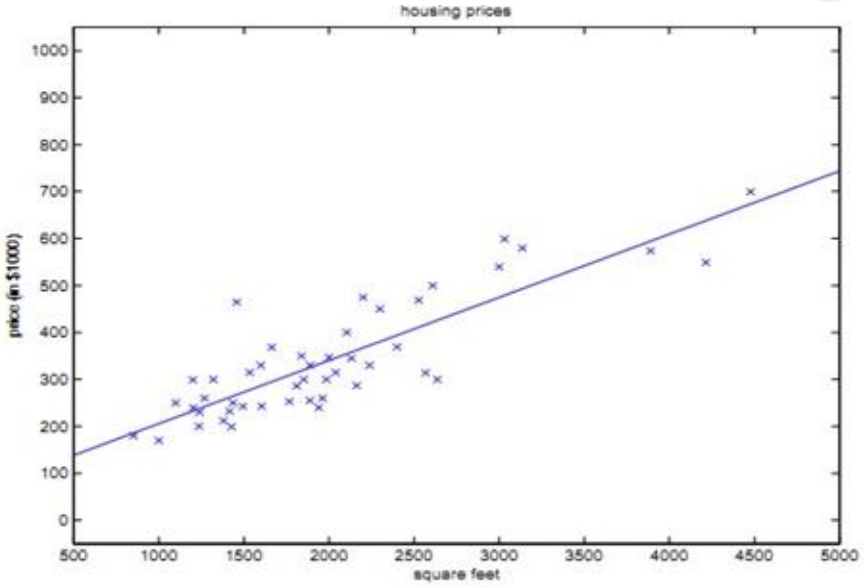
Batch Gradient Descent Algorithm



The ellipses shown above are the contours of a quadratic function. Also shown is the trajectory taken by gradient descent, with was initialized at (48,30). The x's in the figure (joined by straight lines) mark the successive values of θ that gradient descent went through.

When we run batch gradient descent to fit θ on our previous dataset, to learn to predict housing price as a function of living area, we obtain $\theta_0 = 71.27$, $\theta_1 = 0.1345$. If we plot $h_\theta(x)$ as a function of x (area), along with the training data, we obtain the following figure:

Batch Gradient Descent Algorithm



If the number of bedrooms were included as one of the input features as well, we get $\theta_0 = 89.60, \theta_1 = 0.1392, \theta_2 = -8.738$.

The above results were obtained with batch gradient descent. There is an alternative to batch gradient descent that also works very well. Consider the following algorithm:

Incremental Gradient Descent Algorithm

Loop {

for i=1 to m, {

$$\theta_j := \theta_j + \alpha \left(y^{(i)} - h_\theta(x^{(i)})\right) x_j^{(i)} \qquad \text{(for every } j\text{)}$$

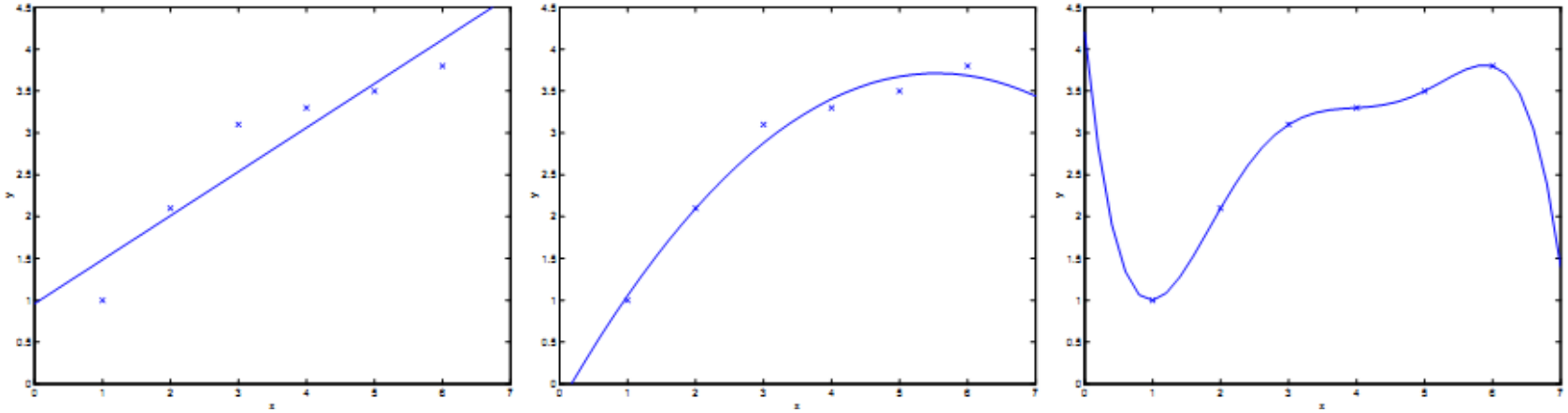
}

}

- In this algorithm, we repeatly run through the training set, and each time we encounter a training example, we update the parameters according to the gradient of the error with respect to that **single training example only**.
- This algorithm is called **stochastic gradient descent** (also incremental gradient descent).

Incremental Gradient Descent Algorithm

- Remember our housing example ?
- Consider the problem of predicting y from x ε R. The leftmost figure below shows the result of fitting a $y = \theta_0 + \theta_1 x$set. It can be notice that the data doesn't really lie on the straight line, and so the fitting is not very good.



Incremental Gradient Descent Algorithm

- If we add an extra feature x^2, and fit $y = \theta_0 + \theta_1 x + \theta_2 x^2$; then we obtain a slightly better fit to the data (See middle figure).
- Naively, it might seem that the more features we add, the better fitting is. However, there is a danger in adding too many features. The rightmost figure is the result of fitting a 5-th order polynomial $y = \sum_{j=0}^{5} \theta_j x^j$. We see that even though the fitted curved passes through the data perfectly, we would not expect this to be a very good predictor of, say, housing prices (y) for different living areas (x).
- Underfitting – Structure not captured by the model
- Overfitting ??

Locally Weighted Linear Regression

- Assume there is sufficient training data, in the original linear regression algorithm, to make a prediction at a query point x, we would
  - Fit θ to minimise $\sum_i (y^{(i)} - \theta^T x^{(i)})^2$
  - Output $\theta^T x$
- Locally weighted linear regression (non-parametric)
  - Fit θ to minimise $\sum_i w^{(i)} (y^{(i)} - \theta^T x^{(i)})^2$
  - Output $\theta^T x$
  - $w^{(i)}$ are non-negative valued weights. If $w^{(i)}$ is large for a particular value of *i*, then in picking θ, we will try hard to make $(y^{(i)} - \theta^T x^{(i)})^2$ small. If w(i) is small, then the $(y^{(i)} - \theta^T x^{(i)})^2$ error term will be pretty much ignored in the fit.
  - A fairly standard choice foe the weights is

$$w^{(i)} = \exp\left(-\frac{(x^{(i)} - x)^2}{2\tau^2}\right)$$

Logistic Regression

- All the regression algorithm that we saw just now are for y = continuous value.
- **So, what happen when y = discrete value ???**
- In this case, we know that $y \in \{0, 1\}$olve this, lets change the form for our hypotheses $h_\theta(x)$. We will choose :

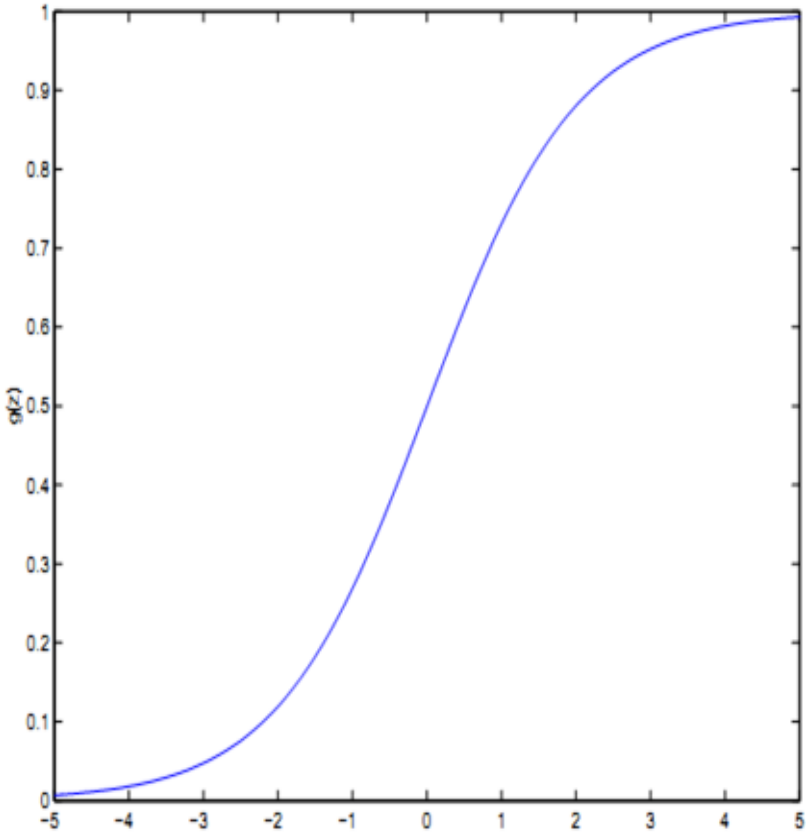
$$\boxed{h_\theta(x) = g(\theta^T x) = \frac{1}{1 + e^{-\theta^T x}}}$$

where $$g(z) = \frac{1}{1 + e^{-z}}$$

- is called the logistic function or the sigmoid function.

Logistic Regression

- The plot showing g(z) is:
- So, given this logistic regression model, how do we fit θ ??
- Following how we saw least square regression could be derived as the maximum likelihood estimator under a set of assumptions, let endow our classification model with a set of probabilistic assumptions, and then fit the parameters via maximum likelihood.



Logistic Regression

- Let us assume that

$$\begin{aligned} P(y=1 \mid x;\theta) &= h_\theta(x) \\ P(y=0 \mid x;\theta) &= 1-h_\theta(x) \end{aligned}$$

- Note that this can be written more compactly as

$$\boxed{p(y \mid x;\theta) = (h_\theta(x))^y\,(1-h_\theta(x))^{1-y}}$$

- Assuming that the m training examples were generated independently, we can then write down the likelihood of the parameters as

$$\boxed{\begin{aligned} L(\theta) &= p(\vec{y} \mid X;\theta) \\ &= \prod_{i=1}^{m} p(y^{(i)} \mid x^{(i)};\theta) \\ &= \prod_{i=1}^{m} \left(h_\theta(x^{(i)})\right)^{y^{(i)}} \left(1-h_\theta(x^{(i)})\right)^{1-y^{(i)}} \end{aligned}}$$

Logistic Regression

- As before, it will be easier to maximise the log likelihood:

$$\boxed{\begin{aligned}\ell(\theta) &= \log L(\theta) \\ &= \sum_{i=1}^{m} y^{(i)} \log h(x^{(i)}) + (1 - y^{(i)}) \log(1 - h(x^{(i)}))\end{aligned}}$$

- But still how do we maximise the likelihood ?????

SOLUTION :

GRADIENT DESCENT !!!!

Logistic Regression

$$
\begin{aligned}
\frac{\partial}{\partial \theta_j}\ell(\theta) &= \left(y\frac{1}{g(\theta^T x)} - (1-y)\frac{1}{1-g(\theta^T x)}\right)\frac{\partial}{\partial \theta_j}g(\theta^T x) \\
&= \left(y\frac{1}{g(\theta^T x)} - (1-y)\frac{1}{1-g(\theta^T x)}\right)g(\theta^T x)(1-g(\theta^T x)\frac{\partial}{\partial \theta_j}\theta^T x \\
&= \left(y(1-g(\theta^T x)) - (1-y)g(\theta^T x)\right)x_j \\
&= \left(y - h_\theta(x)\right)x_j
\end{aligned}
$$

- Above, we used the fact thatr g'(z) = g(z)(1-g(z)). This therefore gives us the stochastic gradient descent rule:

$$\theta_j := \theta_j + \alpha\left(y^{(i)} - h_\theta(x^{(i)})\right)x_j^{(i)}$$

THANK YOU